Tour Blog

Sri Lanka Travel Information Platform

Technical Report

A comprehensive explanation of every component
of the full-stack web application

Author: UpekaFernando

Repository: `UpekaFernando/tour_blog_info_final`

Frontend: React 18 + Vite + Material-UI

Backend: Node.js + Express + Sequelize

Database: MySQL 8.0

DevOps: Docker, Jenkins, Terraform, AWS

Date: March 23, 2026

Contents

Chapter 1

Project Overview

1.1 Purpose and Goals

Tour Blog is a full-stack web application designed for exploring, sharing, and managing information about tourist destinations in **Sri Lanka**. The platform enables:

- **Destination Discovery** — Users can browse an interactive map and a curated list of Sri Lankan destinations organised by district.
- **Community Contributions** — Authenticated users may add new destinations, post comments, and submit star ratings.
- **Local Services Directory** — Hotels, restaurants, tour operators, and transport services are catalogued with pricing tiers and contact details.
- **Travel Guides** — Step-by-step guides covering planning, budget travel, cultural etiquette, visa information, and emergency contacts.
- **Weather Information** — Live weather data via an external API, helping visitors plan their trips.
- **Photo Gallery** — A visual gallery of destination images uploaded by users.

1.2 Technology Stack

Table 1.1 summarises the technologies used across every layer of the system.

Table 1.1: Technology stack

Layer	Technology	Version / Notes
Frontend framework	React	18 (Vite build tool)
UI component library	Material-UI (MUI)	v7
Routing	React Router DOM	v7
HTTP client	Axios	v1.10
Mapping	Mapbox GL / react-map-gl	v3 / v8
Styling	Emotion + styled-components	v11 / v6
Backend framework	Express.js	v4.18
ORM	Sequelize	v6.37
Database	MySQL	8.0
Authentication	JSON Web Tokens	9.0
Password hashing	bcryptjs	2.4
File uploads	Multer	v2
Containerisation	Docker + Docker Compose	latest
Web server	Nginx (Alpine)	production frontend
CI/CD	Jenkins	Declarative Pipeline
Infrastructure	Terraform	1.7.0
Cloud provider	AWS (EC2, RDS, S3)	us-east-1

1.3 High-Level Architecture

The application follows a **three-tier architecture**:

1. **Presentation tier** — React Single-Page Application (SPA) served by Nginx inside a Docker container.
2. **Application tier** — Node.js/Express REST API server running in a separate Docker container, exposing endpoints on port `5000`.
3. **Data tier** — AWS RDS MySQL 8.0 instance (identifier `tour-blog-db`) storing all persistent data.

Both Docker containers run on an AWS EC2 `t3.micro` instance. The frontend is also published as a static website to an **AWS S3 bucket** for high-availability access. A **Jenkins** CI/CD pipeline automates building, testing, and deploying new versions.

1.4 Repository Root Structure

Listing 1.1: Top-level directory layout

```
tour_blog_info_final/
|-- client/                  # React frontend application
|-- server/                  # Node.js/Express backend API
|-- Terraform.tf             # AWS infrastructure definition
|-- docker-compose.yml       # Local development orchestration
|-- Jenkinsfile              # CI/CD pipeline definition
|-- user-data-inline.sh      # EC2 bootstrap script
```

```
|-- cleanup-terraform-state.sh  # Terraform state helper
|-- .dockerignore              # Docker build exclusions
|-- .gitignore                 # Git exclusions
|-- README.md                  # Project readme
```

Chapter 2

Frontend Application (client/)

2.1 Overview

The frontend is a **React 18 Single-Page Application** built with **Vite** as the build tool. It implements the complete user interface: navigation, destination browsing, user authentication, forms for adding/editing content, and an interactive Mapbox map of Sri Lanka.

The `client/` directory contains all source code, configuration, and Docker/Nginx files needed to build and serve the frontend both in development and in production.

2.2 Key Configuration Files

2.2.1 package.json — Dependencies and Scripts

`package.json` declares every JavaScript dependency required by the frontend and defines the npm scripts used for development, building, and linting. Key entries:

Listing 2.1: Frontend npm scripts

```
"dev":     "vite"              # Start Vite dev server (HMR)
"build":   "vite build"        # Production bundle
"lint":    "eslint ."          # Static code analysis
"preview": "vite preview"      # Preview production build
   locally
```

Notable production dependencies include `react`, `react-dom`, `react-router-dom`, `@mui/material`, `axios`, `mapbox-gl`, and `react-map-gl`. The MUI icon package (`@mui/icons-material`) supplies the icon set used throughout the UI.

2.2.2 vite.config.js — Build Tool Configuration

Vite is configured as the module bundler for the React application. The `@vitejs/plugin-react` plugin enables:

- Fast Refresh (Hot Module Replacement) during development.

- JSX transformation without manual React imports.
- Optimised production bundles via Rollup under the hood.

2.2.3 .env and .env.production — Environment Variables

Runtime configuration is separated from code using environment files. Vite exposes only variables prefixed with `VITE_` to the browser bundle.

Listing 2.2: Example .env settings

```
VITE_API_URL=http://localhost:5000/api    # Backend API base
   URL
```

In production (built by Jenkins or Docker), `VITE_API_URL` is overridden via a build argument to point at the AWS EC2 instance: `http://<EC2`$_{PUBLIC_IP}$`>`: 5000/*api*.

2.2.4 eslint.config.js — Code Quality

ESLint is configured with the React plugin and React Hooks plugin to enforce consistent code style, catch common React mistakes (e.g. missing dependency arrays in useEffect), and prevent common accessibility problems.

2.3 Dockerfile_frontend — Multi-Stage Docker Build

The frontend Docker image uses a **two-stage build** to keep the final image small (only Nginx and compiled static files are shipped):

Listing 2.3: Multi-stage Dockerfile_frontend logic

```
# Stage 1 -- Builder
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
ARG VITE_API_URL          # Injected at build time by Jenkins
ENV VITE_API_URL=$VITE_API_URL
RUN npm run build          # Outputs to /app/dist

# Stage 2 -- Production server
FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Why multi-stage? The node:18-alpine image (~300 MB) is only needed to compile the app. The final nginx:alpine image is under 30 MB and serves the compiled static files.

2.4 nginx.conf — Production Web Server

The Nginx configuration is tuned for a React SPA with the following important directives:

- **try_files $uri $uri/ /index.html** – Ensures all client-side routes fall back to index.html so React Router can handle navigation without 404 errors.
- **Gzip compression** (gzip on) – Reduces transfer sizes for HTML, CSS, and JavaScript files.
- **Cache headers** – Hashed asset filenames receive a 1-year Cache-Control: max-age for optimal browser caching.
- **Security headers** – X-Frame-Options: SAMEORIGIN, X-Content-Type-Options: nosniff, and X-XSS-Protection harden the frontend against common attacks.

2.5 Source Code Structure (src/)

Listing 2.4: Frontend source layout

```
src/
|-- App.jsx              # Root component: theme + routing
|-- main.jsx             # ReactDOM.render entry point
|-- App.css / index.css
|-- components/
|   |-- Header.jsx
|   |-- Footer.jsx
|   |-- DestinationCard.jsx
|   `-- SriLankaMap.jsx
|-- pages/               # 16 page-level components
|-- context/
|   `-- AuthContext.jsx
`-- utils/
    `-- api.js
```

2.5.1 main.jsx — Entry Point

main.jsx is the first file executed by Vite. It mounts the React application tree into the #root DOM element declared in index.html:

Listing 2.5: main.jsx

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.jsx'
import './index.css'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
```

```
)
```

React.StrictMode activates additional runtime warnings during development (double-invo
of certain lifecycle methods) without affecting production behaviour.

2.5.2 App.jsx — Application Root

App.jsx is the highest-level React component. It performs three responsibilities:

1. **Theme definition** – A custom MUI theme applies the brand colours (dark cyan #006064 as primary, orange #FF9800 as secondary) site-wide.
2. **Authentication context** – Wraps the entire tree in <AuthProvider> so all descendant components can access login state.
3. **Route definitions** – Uses react-router-dom's <BrowserRouter> and <Routes> to declare all 16 routes of the application.

Listing 2.6: Simplified App.jsx routing structure

```
<BrowserRouter>
  <AuthProvider>
    <ThemeProvider theme={theme}>
      <Header />
      <Routes>
        <Route path="/"                element={<HomePage />} />
        <Route path="/explore"         element={<ExplorePage />}
    />
        <Route path="/destinations/:id" element={<
    DestinationPage />} />
        <Route path="/district/:id"  element={<DistrictPage
    />} />
        <Route path="/add"             element={<
    AddDestinationPage />} />
        <Route path="/edit/:id"        element={<
    EditDestinationPage />} />
        <Route path="/login"           element={<LoginPage />}
    />
        <Route path="/register"        element={<RegisterPage
    />} />
        <Route path="/profile"         element={<ProfilePage />}
     />
        <Route path="/guides"          element={<TravelGuidePage
     />} />
        <Route path="/gallery"         element={<
    PhotoGalleryPage />} />
        <Route path="/services"        element={<
    LocalServicesPage />} />
        <Route path="/weather"         element={<
    WeatherClimatePage />} />
        <Route path="/about"           element={<AboutPage />}
    />
```

```
        <Route path="/destinations"  element={<AllDestinationsPage />} />
        <Route path="*"              element={<NotFoundPage />} />
      </Routes>
      <Footer />
    </ThemeProvider>
  </AuthProvider>
</BrowserRouter>
```

2.5.3 AuthContext.jsx — Authentication State Management

AuthContext implements the React Context API to share authentication state across the entire component tree without prop drilling.

- **State** - currentUser (user object or null), isAuthenticated (boolean), loading (boolean during initial check).
- **Persistence** - On mount, the context reads the JWT token and user object from localStorage to restore sessions after a page refresh.
- **login(userData, token)** - Stores the user and token in both React state and localStorage.
- **logout()** - Clears localStorage and resets state.
- **updateUser(userData)** - Allows the profile page to update the cached user object without re-login.

Any component that needs the current user can call useContext(AuthContext) (or the custom useAuth() hook).

2.5.4 utils/api.js — API Service Layer

All communication with the backend is centralised in api.js. This prevents scattered axios.get(...) calls across components and makes it easy to change the base URL.

Listing 2.7: api.js – Axios instance and helper

```
import axios from 'axios';

const API_URL = import.meta.env.VITE_API_URL || 'http://localhost:5000/api';

const api = axios.create({ baseURL: API_URL });

// Attach JWT token to every request automatically
api.interceptors.request.use(config => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

```
13
14 // Generic helpers
15 export const get  = (url, params) => api.get(url, { params });
16 export const post = (url, data)   => api.post(url, data);
17
18 // Domain-specific helpers
19 export const getDestinations    = ()     => get('/destinations
   ');
20 export const getDestinationById = (id)   => get(`/destinations
   /${id}`);
21 export const createDestination  = (data) => post('/
   destinations', data);
22 export const loginUser          = (data) => post('/users/login
   ', data);
23 export const registerUser       = (data) => post('/users',
   data);
24 // ... and many more
```

The Axios request interceptor automatically injects the Authorization: Bearer <token> header on every outgoing request if a token is found in localStorage, so protected API calls require no special handling at the call-site.

2.6 Page Components (pages/)

The application contains **16 page-level components**. Each page is a React function component rendered when the corresponding route is matched.

File	Description
HomePage.jsx	Landing page with hero banner, featured destinations, and call-to-action buttons.
ExplorePage.jsx	District-based browsing with filter controls; lists all 25 Sri Lankan districts.
DestinationPage.jsx	Detailed view of a single destination: images, description, comments, ratings, and location map.
DistrictPage.jsx	Shows all destinations within a selected district with a district map.
AddDestinationPage.jsx	Multi-field form (title, description, images, coordinates, best time, tips) for authenticated users to add new destinations.
EditDestinationPage.jsx	Pre-filled form allowing destination authors to update or delete their entries.
LoginPage.jsx	Email/password login form with error feedback; redirects on success.

RegisterPage.jsx	Registration form (name, email, password); validates and calls registerUser().
ProfilePage.jsx	Displays user info, allows updating name/password/profile picture.
TravelGuidePage.jsx	Lists travel guides categorised by topic (planning, budget, visa, safety, etc.).
PhotoGalleryPage.jsx	Masonry-style gallery of all destination images.
LocalServicesPage.jsx	Directory of local services filterable by category (restaurant, hotel, transport, tour operator).
WeatherClimatePage.jsx	Displays real-time weather data for Sri Lankan cities.
AboutPage.jsx	Static information page about the platform.
AllDestinationsPage.jsx	Paginated list of all destinations with search and sort.
NotFoundPage.jsx	404 error page shown for unrecognised routes.

2.7 Shared Components (components/)

2.7.1 Header.jsx

The navigation bar renders differently based on screen size (responsive) and authentication state:

- On **desktop**, a horizontal menu bar shows all primary navigation links.
- On **mobile**, a hamburger button opens a slide-in drawer with the same links.
- When a user is **logged in**, an avatar/profile menu is shown with options to view the profile or log out.
- When **logged out**, Login and Register buttons are displayed.

2.7.2 Footer.jsx

A simple site footer with links to key sections, social media icons (from MUI icons), and copyright information.

2.7.3 DestinationCard.jsx

A reusable MUI Card component that displays a destination's thumbnail image, title, district, short description, and a link to the full destination page. Used in ExplorePage and AllDestinationsPage.

2.7.4 SriLankaMap.jsx

An interactive map powered by mapbox-gl via the react-map-gl React wrapper. It:

- Renders a styled base map centred on Sri Lanka.
- Plots markers for each destination using the latitude/longitude from the database.
- Shows a popup with the destination name and a link on marker click.

Chapter 3

Backend Server (server/)

3.1 Overview

The backend is a **Node.js / Express.js** REST API that handles all business logic, data persistence, authentication, and file uploads. It follows the classic **MVC (Model-View-Controller)** pattern adapted for a REST API: Models define data structure and database interaction; Controllers contain business logic; Routes map HTTP endpoints to controller functions.

The server listens on port **5000** and exposes all endpoints under the /api prefix.

3.2 server.js — Application Entry Point

server.js bootstraps the entire backend application. Its responsibilities are:

1. **Load environment variables** - dotenv.config() reads .env before anything else, making database credentials and the JWT secret available via process.env.
2. **Create Express app** - Initialises Express and registers global middleware.
3. **CORS configuration** - Allows cross-origin requests from any origin (origin: '*') during development, with credentials enabled. This is necessary because the frontend and backend run on different origins (ports or domains).
4. **Body parsing** - express.json() and express.urlencoded() parse incoming JSON and form-encoded request bodies.
5. **Cache-Control headers** - A custom middleware adds no-store, no-cache headers to all /api responses, preventing browsers from caching API responses and serving stale data.
6. **Static file serving** - express.static('uploads') makes the /uploads directory publicly accessible so the frontend can display user-uploaded images.
7. **Route mounting** - Each feature area has its own router mounted at a sub-path (e.g. /api/users, /api/destinations).

8. **Database connection with retry** – Connects to MySQL using Sequelize with up to 3 retries and a 5-second backoff. If the database is unavailable, the server still starts and returns errors on database-dependent routes.
9. **Server startup** – app.listen(PORT) starts the HTTP server on the configured port.

Listing 3.1: server.js – Startup sequence (simplified)

```
const app = express();

// Middleware
app.use(cors({ origin: '*', credentials: true }));
app.use(express.json());
app.use('/uploads', express.static(path.join(__dirname, '
   uploads')));

// Routes
app.use('/api/users',          userRoutes);
app.use('/api/destinations',   destinationRoutes);
app.use('/api/districts',      districtRoutes);
app.use('/api/comments',       commentRoutes);
app.use('/api/ratings',        ratingRoutes);
app.use('/api/local-services',localServiceRoutes);
app.use('/api/travel-guides', travelGuideRoutes);
app.use('/api/weather',        weatherRoutes);

// Start server
connectDB().then(() => {
  app.listen(5000, () => console.log('Server running on port
   5000'));
});
```

3.3 config/database.js — Database Configuration

This module exports a configured Sequelize instance that connects to MySQL using credentials from environment variables:

Listing 3.2: config/database.js

```
const sequelize = new Sequelize(
  process.env.DB_NAME,
  process.env.DB_USER,
  process.env.DB_PASSWORD,
  {
    host:    process.env.DB_HOST,
    dialect: 'mysql',
    port:    process.env.DB_PORT || 3306,
    logging: false,              // Suppress SQL logs in
   production
    pool: {
      max: 10,  min: 0,
```

```
      acquire: 30000,
      idle: 10000
    }
  }
);
```

The **connection pool** allows multiple simultaneous database queries without opening a new connection for every request, improving performance under load.

3.4 Database Models (models/)

Sequelize models define the database schema using JavaScript classes. Each model maps to a MySQL table. The models/index.js file imports all models and defines the **associations (relationships)** between them.

3.4.1 User Model

The User model stores account information and handles password security:

- **Fields:** name (STRING), email (STRING, unique), password (STRING), profilePicture (STRING, nullable), isAdmin (BOOLEAN, default false).
- **Password hashing hooks:** A beforeCreate and beforeUpdate Sequelize lifecycle hook automatically hashes the password using bcryptjs with 10 salt rounds whenever a user is created or their password is changed. This means the plain-text password is *never* stored in the database.
- **matchPassword()** - An instance method that compares a plain-text candidate password against the stored hash using bcryptjs.compare(), used during login.

3.4.2 District Model

Represents the 25 districts of Sri Lanka:

- **Fields:** name (STRING, unique), description (TEXT), mapCoordinates (JSON -- lat/lng bounds), imageUrl (STRING), province (STRING).
- Districts are the top-level geographical grouping. Each district can contain many destinations.

3.4.3 Destination Model

The core entity of the application:

- **Fields:** title, description (TEXT), images (JSON array of file paths), location (JSON with lat/lng), bestTimeToVisit (STRING), travelTips (TEXT).
- **Foreign keys:** districtId (belongs to District, CASCADE delete -- if the district is deleted, all its destinations are deleted too), authorId (belongs to User).

- Images are stored as an array of file paths, allowing multiple photos per destination.

3.4.4 Comment Model

Stores user comments on destinations:

- **Fields:** content (TEXT, required).
- **Relations:** belongs to User and Destination. If the parent destination is deleted, all its comments are deleted (CASCADE).

3.4.5 Rating Model

Stores 1-5 star ratings submitted by users for destinations:

- **Fields:** value (INTEGER, validated: min 1, max 5).
- A user can rate the same destination only once (unique constraint on userId + destinationId).

3.4.6 LocalService Model

A rich model for the local services directory:

- **category ENUM:** restaurant | hotel | transport | tour-operator | other.
- **priceRange ENUM:** $ | $$ | $$$ | $$$$ (budget to luxury).
- **JSON arrays:** features, specialties, amenities, services allow rich structured data without needing separate join tables.
- **isVerified** flag to distinguish officially verified listings from user-submitted ones.

3.4.7 TravelGuide Model

Stores user-contributed and official travel guides:

- **category ENUM:** planning | transport | budget | cultural | safety | visa | emergency | language | other.
- **JSON tags** array for flexible tagging.
- **viewCount / helpfulCount** counters to surface the most useful guides.
- **isOfficial / isVerified** flags to distinguish platform-curated content from community submissions.

3.4.8 Model Relationships

Listing 3.3: models/index.js – Association definitions

```
// District -> Destination: one district has many destinations
District.hasMany(Destination, { foreignKey: 'districtId',
  onDelete: 'CASCADE' });
```

```
Destination.belongsTo(District, { foreignKey: 'districtId' });

// User -> Destination: a user can author many destinations
User.hasMany(Destination,        { foreignKey: 'authorId', as: 'destinations' });
Destination.belongsTo(User,      { foreignKey: 'authorId', as: 'author' });

// User/Destination -> Comments
User.hasMany(Comment,            { foreignKey: 'userId' });
Destination.hasMany(Comment,     { foreignKey: 'destinationId', onDelete: 'CASCADE' });
Comment.belongsTo(User,          { foreignKey: 'userId' });
Comment.belongsTo(Destination,   { foreignKey: 'destinationId' });

// User/Destination -> Ratings
User.hasMany(Rating,             { foreignKey: 'userId' });
Destination.hasMany(Rating,      { foreignKey: 'destinationId', onDelete: 'CASCADE' });

// User -> LocalServices
User.hasMany(LocalService,       { foreignKey: 'userId' });

// User -> TravelGuides
User.hasMany(TravelGuide,        { foreignKey: 'userId', as: 'guides' });
```

3.5 Controllers (controllers/)

Controllers contain the business logic for each feature area. They receive (req, res) from Express routes, interact with Sequelize models, and send back JSON responses.

3.5.1 userController.js

- **registerUser** – Validates email uniqueness, creates a User record (password is hashed automatically by the model hook), returns the new user and a JWT token.
- **loginUser** – Finds user by email, calls matchPassword() to verify the password, returns user data and a signed JWT (30-day expiry).
- **getUserProfile** – Returns the authenticated user's profile (protected by authMiddleware).
- **updateUserProfile** – Allows updating name, password, and profile picture (file upload handled by Multer middleware).

3.5.2 destinationController.js

- **getDestinations** – Returns all destinations with associated District and author User data (eager loading with Sequelize's include). Supports filtering by districtId query parameter.
- **getDestinationById** – Returns a single destination with its comments (including commenter name), average rating, and associated district.
- **createDestination** – Creates a new destination. Images are saved to disk by Multer and their paths stored as a JSON array.
- **updateDestination** – Updates an existing destination; checks that the requester is the author or an admin.
- **deleteDestination** – Deletes a destination and cascades to comments and ratings.
- **removeDestinationImage** – Removes a single image from a destination's images array by index.

3.5.3 districtController.js

Provides simple read operations for the District table: list all districts and retrieve a single district by ID (including associated destinations).

3.5.4 commentController.js

- **addComment** – Authenticated users can post a comment on a destination.
- **updateComment** – Users can edit their own comments.
- **deleteComment** – Users can delete their own comments (admins can delete any comment).
- **getCommentsByDestination** – Retrieves all comments for a destination, including the author's name and profile picture.

3.5.5 ratingController.js

- **submitRating** – Creates or updates the authenticated user's rating for a destination (upsert behaviour).
- **getAverageRating** – Computes and returns the average rating for a destination.

3.5.6 localServiceController.js

Full CRUD for the local services directory. Supports rich filtering: by category, by district, by price range, and free-text search.

3.5.7 travelGuideController.js

Full CRUD for travel guides. Supports filtering by category, sorting by view count or helpful count, and incrementing the viewCount when a guide is read.

3.5.8 weatherController.js

Delegates to weatherService.js to fetch live weather data and returns it to the frontend. Caches results to avoid excessive external API calls.

3.6 Routes (routes/)

Route files use Express Router to define HTTP method + path pairs and connect them to controller functions. Some routes apply the authMiddleware and/or uploadMiddleware.

Table 3.1: Key API endpoints

Method + Path	Controller function	Auth?
POST /api/users	registerUser	No
POST /api/users/login	loginUser	No
GET /api/users/profile	getUserProfile	Yes
PUT /api/users/profile	updateUserProfile	Yes + Upload
GET /api/destinations	getDestinations	No
GET /api/destinations/:id	getDestinationById	No
POST /api/destinations	createDestination	Yes + Upload
PUT /api/destinations/:id	updateDestination	Yes
DELETE /api/destinations/:id	deleteDestination	Yes
GET /api/districts	getAllDistricts	No
GET /api/districts/:id	getDistrictById	No
POST /api/comments	addComment	Yes
PUT /api/comments/:id	updateComment	Yes
DELETE /api/comments/:id	deleteComment	Yes
POST /api/ratings	submitRating	Yes
GET /api/local-services	getLocalServices	No
GET /api/travel-guides	getTravelGuides	No
GET /api/weather/:city	getWeather	No

3.7 Middleware (middleware/)

3.7.1 authMiddleware.js — JWT Verification

This middleware protects routes that require a logged-in user:

Listing 3.4: authMiddleware.js

```
const protect = async (req, res, next) => {
  let token;
  if (req.headers.authorization?.startsWith('Bearer')) {
    token = req.headers.authorization.split(' ')[1];
  }
  if (!token) {
    return res.status(401).json({ message: 'Not authorised, no
    token' });
```

```
  }
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = await User.findByPk(decoded.id,
                  { attributes: { exclude: ['password'] } });
    next();
  } catch (error) {
    res.status(401).json({ message: 'Not authorised, token
   failed' });
  }
};
```

The middleware reads the token from the Authorization header, verifies it against the JWT_SECRET environment variable, and attaches the decoded user object to req.user so downstream controllers can access it.

3.7.2 uploadMiddleware.js — File Upload Handling

Built on **Multer**, this middleware handles multipart/form-data file uploads:

- **Storage** – Files are saved to the server/uploads/ directory with a unique timestamp-based filename to prevent collisions.
- **File filter** – Only image MIME types (image/jpeg, image/png, image/gif, image/webp are accepted; other file types are rejected with a 400 error.
- **Size limit** – Individual files are capped at 5 MB.
- Two upload configurations are exported: uploadSingle (profile pictures) and uploadMultiple (up to 10 destination images).

3.8 services/weatherService.js — External Weather Integration

weatherService.js encapsulates all interaction with the external weather API (e.g. OpenWeatherMap). It:

- Constructs API requests with the city name and API key (from environment variables).
- Normalises the API response into a consistent internal format.
- Implements a simple in-memory cache to avoid repeated API calls for the same city within a short window.

3.9 Dockerfile_backend

Listing 3.5: Backend Dockerfile (simplified)

```
FROM node:18-alpine
WORKDIR /app
```

```
COPY package*.json ./
RUN npm install --production   # Only production dependencies
COPY . .
RUN mkdir -p uploads            # Ensure uploads directory
   exists
EXPOSE 5000
HEALTHCHECK --interval=30s --timeout=10s \
  CMD wget -qO- http://localhost:5000/api/test || exit 1
CMD ["node", "server.js"]
```

The HEALTHCHECK directive allows Docker (and Docker Compose) to detect if the backend has crashed or become unresponsive and restart it automatically.

Chapter 4

Database Design

4.1 Overview

The application uses **MySQL 8.0** hosted on AWS RDS. The schema is managed by **Sequelize ORM**, which synchronises the JavaScript model definitions to the database tables. The database name is `tour_blog`.

4.2 Entity-Relationship Summary

Figure description (textual ER diagram):

Listing 4.1: Textual ER diagram

```
[User] ---1:N---> [Destination]  (authorId)
[User] ---1:N---> [Comment]      (userId)
[User] ---1:N---> [Rating]       (userId)
[User] ---1:N---> [LocalService] (userId)
[User] ---1:N---> [TravelGuide]  (userId)

[District] ---1:N---> [Destination]  (districtId, CASCADE)

[Destination] ---1:N---> [Comment] (destinationId, CASCADE)
[Destination] ---1:N---> [Rating]  (destinationId, CASCADE)
```

4.3 Table Definitions

Table 4.1: users table

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Unique identifier
name	VARCHAR(255)	NOT NULL	Display name
email	VARCHAR(255)	NOT NULL, UNIQUE	Login email
password	VARCHAR(255)	NOT NULL	bcrypt hash
profilePicture	VARCHAR(255)	NULL	Upload file path
isAdmin	BOOLEAN	DEFAULT false	Admin flag
createdAt	DATETIME		Auto-managed
updatedAt	DATETIME		Auto-managed

Table 4.2: districts table

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	
name	VARCHAR(255)	NOT NULL, UNIQUE	District name
description	TEXT		
mapCoordinates	JSON		Lat/lng bounds
imageUrl	VARCHAR(255)		
province	VARCHAR(255)		Province name

Table 4.3: destinations table

Column	Type	Constraints	Description
id	INT	PK	
title	VARCHAR(255)	NOT NULL	
description	TEXT	NOT NULL	
images	JSON		Array of paths
location	JSON		{lat, lng}
bestTimeToVisit	VARCHAR(255)		
travelTips	TEXT		
districtId	INT	FK districts	
authorId	INT	FK users	

4.4 Data Seeding

The server directory contains several database utility scripts:

- **createDatabase.js** - Creates the tour_blog schema if it does not exist.
- **setupNewDatabase.js** - Runs schema synchronisation (sequelize.sync()).

- **seed.js / seedNewDatabase.js** - Populates the database with sample districts, destinations, and travel guides for demonstration purposes.
- **testConnection.js / mysqlCheck.js** - Utility scripts to verify that the application can reach the database before deployment.

Chapter 5

Docker and Containerisation

5.1 Overview

The entire application stack is containerised using **Docker**, enabling consistent behaviour across development, CI/CD, and production environments. All services are orchestrated locally using **Docker Compose**.

5.2 docker-compose.yml

The Compose file defines three services:

Listing 5.1: docker-compose.yml services

```
services:
  mysql:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword
      MYSQL_DATABASE: tour_blog
      MYSQL_USER: admin
      MYSQL_PASSWORD: password
    ports: ["3306:3306"]
    volumes: [mysql_data:/var/lib/mysql]
    healthcheck:
      test: ["CMD", "mysqladmin", "ping"]
      interval: 30s
      retries: 3

  backend:
    build:
      context: ./server
      dockerfile: Dockerfile_backend
    ports: ["5000:5000"]
    environment:
      DB_HOST: mysql
      DB_USER: admin
      DB_PASSWORD: password
```

```
      JWT_SECRET: dev-secret
    depends_on:
      mysql: { condition: service_healthy }
    volumes: [./server/uploads:/app/uploads]

  frontend:
    build:
      context: ./client
      dockerfile: Dockerfile_frontend
      args:
        VITE_API_URL: http://localhost:5000/api
    ports: ["80:80"]
    depends_on: [backend]
```

Key points:

- **Health checks** – MySQL reports healthy only after accepting connections; the backend container waits for this (condition: service_healthy) to avoid startup race conditions.
- **Named volume** (mysql_data) – Persists MySQL data across container restarts so database contents survive docker compose down.
- **Uploads volume** – Mounts ./server/uploads into the backend container so uploaded files persist and are accessible on the host.
- **Service networking** – Docker Compose creates a default bridge network; the backend connects to MySQL using the service name mysql as the hostname.

5.3 .dockerignore

The .dockerignore file excludes unnecessary files from the Docker build context (e.g. node_modules, .git, dist) to speed up image builds and reduce image sizes.

Chapter 6

CI/CD Pipeline (Jenkinsfile)

6.1 Overview

The Jenkins Declarative Pipeline automates the end-to-end build-and-deploy workflow. Every push to the repository triggers the pipeline, which:

1. Prepares the build environment.
2. Builds Docker images for both backend and frontend.
3. Pushes images to **Docker Hub** (under the upeka2002 account).
4. Installs Terraform if not already present.
5. Runs Terraform to apply any infrastructure changes on AWS.

6.2 Environment Variables

Listing 6.1: Jenkinsfile environment block

```
environment {
  DOCKERHUB_REPO    = "upeka2002"
  BACKEND_IMAGE     = "upeka2002/tourblog-backend"
  FRONTEND_IMAGE    = "upeka2002/tourblog-frontend"
  TF_VERSION        = "1.7.0"
}
```

6.3 Pipeline Stages Explained

6.3.1 Stage 1: Prepare

Checks out the latest source code and generates a short **Git commit SHA** to use as the Docker image tag. Using the commit SHA (instead of latest) ensures every image is traceable to the exact code version:

Listing 6.2: Prepare stage

```
stage('Prepare') {
  steps {
    checkout scm
    script {
      IMAGE_TAG = sh(returnStdout: true,
                     script: 'git rev-parse --short HEAD').
   trim()
      echo "Building image tag: ${IMAGE_TAG}"
    }
  }
}
```

6.3.2 Stage 2: Build Backend

Builds the backend Docker image from ./server/Dockerfile_backend, tagging it with both the commit SHA and latest:

Listing 6.3: Build Backend stage

```
stage('Build Backend') {
  steps {
    dir('server') {
      sh """
        docker build -f Dockerfile_backend \
          -t ${BACKEND_IMAGE}:${IMAGE_TAG} \
          -t ${BACKEND_IMAGE}:latest .
      """
    }
  }
}
```

6.3.3 Stage 3: Build Frontend

Builds the frontend Docker image. The **AWS EC2 public IP** is injected as a build argument so Vite embeds the correct API URL into the compiled JavaScript bundle:

Listing 6.4: Build Frontend stage

```
stage('Build Frontend') {
  steps {
    dir('client') {
      sh """
        docker build \
          --build-arg VITE_API_URL=http://<EC2_PUBLIC_IP
   >:5000/api \
          -f Dockerfile_frontend \
          -t ${FRONTEND_IMAGE}:${IMAGE_TAG} \
          -t ${FRONTEND_IMAGE}:latest .
      """
    }
  }
}
```

6.3.4 Stage 4: Push to Docker Hub

Authenticates with Docker Hub using Jenkins credentials and pushes both tagged images:

Listing 6.5: Push to Docker Hub stage

```
stage('Push to Docker Hub') {
  steps {
    withCredentials([usernamePassword(
      credentialsId: 'dockerhub-credentials',
      usernameVariable: 'DOCKER_USER',
      passwordVariable: 'DOCKER_PASS'
    )]) {
      sh """
        echo "$DOCKER_PASS" | docker login -u "$DOCKER_USER"
   --password-stdin
        docker push ${BACKEND_IMAGE}:${IMAGE_TAG}
        docker push ${BACKEND_IMAGE}:latest
        docker push ${FRONTEND_IMAGE}:${IMAGE_TAG}
        docker push ${FRONTEND_IMAGE}:latest
        docker logout
      """
    }
  }
}
```

Security note: The password is never echoed; -password-stdin reads it from stdin.

6.3.5 Stage 5: Install Terraform

Checks if Terraform is installed; if not, downloads version 1.7.0 from the official HashiCorp releases:

Listing 6.6: Install Terraform stage

```
stage('Install Terraform') {
  steps {
    sh """
      if ! command -v terraform &>/dev/null; then
        wget https://releases.hashicorp.com/terraform/${
   TF_VERSION}/
             terraform_${TF_VERSION}_linux_amd64.zip
        unzip terraform_${TF_VERSION}_linux_amd64.zip
        mv terraform /usr/local/bin/
      fi
      terraform version
    """
  }
}
```

6.3.6 Stages 6–9: Terraform Init, State Cleanup, Plan, Apply

These stages use the AWS credentials stored in Jenkins (aws-access-key-id and aws-secret-access-key) to authenticate with AWS and apply infrastructure changes:

- **Terraform Init** – Downloads the AWS provider plugin and initialises the working directory.
- **State Cleanup** – Removes stale resource entries from the Terraform state file to avoid errors with pre-existing resources.
- **Terraform Plan** – Generates an execution plan (tfplan) showing exactly what changes will be made to AWS.
- **Terraform Apply** – Applies the plan automatically (-auto-approve), updating S3 bucket configuration.

Chapter 7

Infrastructure as Code (Terraform.tf)

7.1 Overview

Terraform.tf defines and manages the **AWS cloud infrastructure** required to run the Tour Blog platform in production. It uses the **HashiCorp Configuration Language (HCL)** and the hashicorp/aws provider (~5.0).

A key design decision is that Terraform **references existing AWS resources** (by their IDs) rather than creating them from scratch. This prevents accidental destruction of live infrastructure while still allowing Terraform to manage configuration (e.g. S3 policies).

7.2 Provider Configuration

Listing 7.1: Provider block

```
terraform {
  required_version = ">= 1.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region  # Default: us-east-1
}
```

The AWS credentials are supplied at runtime via environment variables (AWS_ACCESS_KEY_ and AWS_SECRET_ACCESS_KEY) injected by Jenkins, keeping secrets out of the repository.

7.3 Referenced Existing Resources

The following resources are referenced using data sources but **not** managed (not created or destroyed) by Terraform:

- **EC2 instance** (<EC2$_I NSTANCE_I D >)¯$Ubuntu 22.04 LTS$ t3.micro $instance running the Docker$
- **S3 buckets** - tour-blog-frontend-<ACCOUNT$_I D >$ and tour-blog-deploy-<ACCOUNT$_I D >$.

7.4 Managed Resources

Terraform actively manages three S3 resources:

1. **S3 Website Configuration** - Configures the frontend bucket as a static website with index.html as both index and error document, enabling React Router to serve all routes.
2. **S3 Public Access Block** - Disables all four AWS public-access blocks on the frontend bucket, allowing the bucket policy to grant public read access.
3. **S3 Bucket Policy** - Attaches an IAM policy granting s3:GetObject to all principals ("*") on all objects in the bucket, making static assets publicly readable.

7.5 Input Variables

Table 7.1: Terraform input variables

Variable	Type	Default	Sensitive
aws_region	string	us-east-1	No
project_name	string	tour-blog	No
ec2_ami_id	string	<AMI$_I D >$	No
ec2_instance_type	string	t3.micro	No
ssh_public_key	string	(empty)	Yes
db_engine_version	string	8.0.40	No
db_instance_class	string	db.t3.micro	No
db_allocated_storage	number	20	No
db_name	string	tour_blog	No
db_username	string	admin	Yes
db_password	string	(empty)	Yes

7.6 Outputs

After a terraform apply, the following values are printed, allowing other tools (e.g. the Jenkins deploy stage) to discover the infrastructure endpoints dynamically:

Table 7.2: Terraform outputs

Output	Value
ec2_public_ip	Public IP of the backend server
ec2_instance_id	AWS instance ID
ec2_ssh_command	Ready-to-use SSH command
rds_endpoint	Full RDS connection string (host:port)
rds_address	RDS hostname only
database_name	`tour_blog`
backend_api_url	`http://{ec2_ip}:5000/api`
s3_frontend_url	S3 static website URL

7.7 user-data-inline.sh — EC2 Bootstrap Script

This Bash script is executed once when the EC2 instance first launches (via AWS User Data). It automates the entire server setup:

1. System update (apt-get update && upgrade)
2. Install Node.js 20.x from NodeSource
3. Install PM2 globally (npm install -g pm2) -- process manager
4. Install MySQL client, Git, unzip
5. Download and install AWS CLI v2
6. Create /app directory with correct permissions
7. Generate .env template for the backend
8. Create a deployment shell script that pulls Docker images and restarts containers

7.8 cleanup-terraform-state.sh

A utility script for Terraform state management. It removes stale resource references from the local Terraform state file (terraform.tfstate), preventing "resource already exists" errors when Terraform tries to manage resources that were created outside of Terraform. This is particularly useful during initial setup or after manual AWS console changes.

Chapter 8

Authentication and Security

8.1 Authentication Flow

The application uses **stateless JWT (JSON Web Token) authentication**:

1. User submits email and password via POST /api/users/login.
2. Backend finds the user by email, calls user.matchPassword(plaintext) which calls bcryptjs.compare().
3. If the password matches, the server calls jwt.sign({ id, name, email }, JWT_SECRET, { expiresIn: '30d' }) to produce a signed token.
4. The token is returned to the frontend in the response body.
5. The frontend stores the token in localStorage and includes it as Authorization: Bearer <token> on every subsequent request.
6. The authMiddleware verifies the token signature on protected routes.

8.2 Password Security

Passwords are hashed using **bcryptjs** with a cost factor (salt rounds) of **10**. This means:

- Plain-text passwords are **never stored** in the database.
- Each password hash is unique due to random salt, preventing rainbow table attacks.
- The cost factor of 10 means ~1024 hash iterations, making brute-force attacks computationally expensive.

8.3 Security Headers (Nginx)

The Nginx configuration adds HTTP security headers to all frontend responses:

- `X-Frame-Options: SAMEORIGIN -- Prevents clickjacking by blocking the page from being embedded in iframes on other domains.`
- `X-Content-Type-Options: nosniff -- Prevents MIME-type sniffing attacks.`
- `X-XSS-Protection: 1; mode=block -- Enables the browser's built-in XSS filter.`

8.4 Security Considerations and Recommendations

Table 8.1: Security assessment

Area	Status	Notes
Password hashing	Implemented	bcryptjs, 10 rounds
JWT authentication	Implemented	30-day expiry
CORS	Partially	All origins allowed (tighten for production)
HTTPS	Not shown	Recommend AWS CloudFront + ACM
Rate limiting	Missing	Recommend express-rate-limit
Input validation	Basic	Add express-validator
SQL injection	Protected	Sequelize ORM parameterises queries
XSS (frontend)	Partial	React escapes JSX; add CSP header

Chapter 9

Deployment Workflow

9.1 End-to-End Deployment Sequence

1. **Developer** pushes code to the GitHub repository.
2. **GitHub Webhook** triggers the Jenkins pipeline.
3. **Jenkins (Stage: Prepare)** checks out code, computes commit SHA for image tagging.
4. **Jenkins (Stage: Build Backend)** builds upeka2002/tourblog-backend:{sha}.
5. **Jenkins (Stage: Build Frontend)** builds upeka2002/tourblog-frontend:{sha} with the EC2 IP embedded.
6. **Jenkins (Stage: Push)** authenticates with Docker Hub and pushes both images.
7. **Jenkins (Stage: Terraform Apply)** updates S3 bucket configuration.
8. **Deploy script on EC2** pulls the new latest images and restarts Docker containers.
9. **Health check** confirms the backend API is responding.

9.2 Local Development

Listing 9.1: Starting the application locally

```
# Clone the repository
git clone https://github.com/UpekaFernando/
   tour_blog_info_final.git
cd tour_blog_info_final

# Start all services (MySQL + backend + frontend)
docker compose up -d --build

# View backend logs
docker compose logs -f backend

# Stop all services
```

```
12 docker compose down
```

After docker compose up, the application is accessible at:

- Frontend: http://localhost:80
- Backend API: http://localhost:5000/api

9.3 Environment Variables Reference

Table 9.1: Required environment variables

Variable	Service	Description
DB_HOST	Backend	MySQL hostname
DB_USER	Backend	MySQL username
DB_PASSWORD	Backend	MySQL password
DB_NAME	Backend	Database name (`tour_blog`)
DB_PORT	Backend	MySQL port (default 3306)
JWT_SECRET	Backend	Secret for signing JWT tokens
NODE_ENV	Backend	`development` or `production`
PORT	Backend	Server port (default 5000)
VITE_API_URL	Frontend (build)	Backend API base URL

Chapter 10

Summary and Conclusions

10.1 What Has Been Built

The Tour Blog platform is a **production-ready, cloud-deployed full-stack web application** covering the complete spectrum of modern software engineering:

- A **React SPA** with Material Design UI, client-side routing, interactive mapping, and responsive layout.
- A **Node.js/Express REST API** with MVC architecture, JWT authentication, file uploads, and comprehensive CRUD endpoints for 6 domain entities.
- A **MySQL relational database** with 7 interconnected tables managed by the Sequelize ORM.
- **Docker containerisation** for consistent environments and simplified deployment.
- A **Jenkins CI/CD pipeline** that automates the entire build → push → deploy cycle.
- **Terraform infrastructure-as-code** managing AWS resources (EC2, RDS, S3).

10.2 Key Design Decisions

1. **Separation of concerns** - Frontend and backend are completely independent services communicating only via HTTP API, enabling independent scaling and deployment.
2. **Environment-based configuration** - All sensitive values (passwords, secrets, API keys) are injected via environment variables, never hard-coded.
3. **Stateless authentication** - JWT tokens allow horizontal scaling of the backend without shared session storage.
4. **Terraform data sources** - Referencing existing AWS resources (rather than recreating them) protects production data from accidental destruction.
5. **Docker multi-stage builds** - Keeps production images small and secure by excluding development tooling.

10.3 Future Improvements

- **HTTPS** – Configure AWS CloudFront with an ACM certificate in front of both the EC2 instance and S3 bucket.
- **CORS hardening** – Restrict allowed origins to the specific frontend domain.
- **Rate limiting** – Add express-rate-limit to prevent API abuse.
- **Image storage** – Move uploaded images from local disk to AWS S3 for durability and scalability.
- **Testing** – Add unit tests (Jest) for controllers and integration tests (Supertest) for API endpoints.
- **Monitoring** – Integrate AWS CloudWatch metrics and alerts.
- **Database migrations** – Use Sequelize migrations (instead of sync()) for controlled schema changes in production.